

Real-Time Chat Application – Workflow & API Documentation

Overview

This web application provides **real-time messaging** between users. It combines RESTful HTTP APIs for authentication and data retrieval with **Socket.IO** for instant message delivery and online user tracking. The frontend uses state management and modern UI libraries, while the backend coordinates persistence, authentication, and socket communication for a seamless user experience. Deployment is handled via **Render**, ensuring scalable hosting with environment variable support.

Tech Stack

Frontend

- **React** – component-based UI framework
- **Zustand** – lightweight state management for Auth Store and Chat Store
- **Tailwind CSS** – utility-first styling
- **DaisyUI** – Tailwind component library for consistent UI elements

Backend

- **Node.js + Express** – REST API server
- **Socket.IO** – real-time communication layer
- **MongoDB (Mongoose)** – database for users and messages
- **Bcrypt** – secure password hashing for authentication

Integrations

- **Arcjet** – rate limiting and bot protection middleware
- **Resend** – transactional email service for onboarding and notifications
- **Cloudinary** – media storage and image hosting

Deployment

- **Render** – cloud hosting platform for backend and frontend services

Version Control

- **GitHub** – repository hosting, collaboration, and version control

Workflow

1. Application Initialization

- **User → Browser:** Open app
- **Browser → Auth Store:** `checkAuth()`
- **Auth Store → Backend API:** `GET /api/auth/auth-check`
- **Backend API → Auth Store:** `200 OK + user`
- **Auth Store → Browser:** Set `authUser`
- **Auth Store → Backend Socket:** Connect socket with credentials
- **Backend Socket → Auth Store:** Validate user via `socketAuthMiddleware`
- **Backend Socket → Browser:** Emit `getOnlineUsers`

2. Authentication (Login / Signup)

- **User → Browser:** Login or signup
- **Browser → Auth Store:** `login()` / `signup()`
- **Auth Store → Backend API:** `POST /api/auth/login` or `POST /api/auth/signup`
- **Backend API → Auth Store:** `200 OK + user`
- **Auth Store → Browser:** Set `authUser`
- **Auth Store → Backend Socket:** Connect socket

(Passwords are securely hashed with **Bcrypt** before storage.)

3. Contacts & Chat Partners

- **Browser → Chat Store:** `getAllContacts()`
- **Chat Store → Backend API:** `GET /api/messages/contacts`
- **Backend API → Chat Store:** Return contacts list
- **Browser → Chat Store:** `getAllChatPartners()`
- **Chat Store → Backend API:** `GET /api/messages/chats`
- **Backend API → Chat Store:** Return chat partners

4. Selecting a Chat

- **User → Browser:** Select chat
- **Browser → Chat Store:** `getMessagesByUserId(userId)`
- **Chat Store → Backend API:** `GET /api/messages/:id`
- **Backend API → Chat Store:** Return message history

5. Sending a Message

- **User → Browser:** Send message
- **Browser → Chat Store:** sendMessage(messageData)
- **Chat Store → Chat Store:** Optimistic UI update
- **Chat Store → Backend API:** POST /api/messages/send/:id
- **Backend API → Backend Socket:** Lookup receiver socket ID
- **Backend Socket → Receiver Browser:** Emit newMessage
- **Receiver Browser → Chat Store:** Update messages

6. Logout

- **User → Browser:** Logout
- **Browser → Auth Store:** logout()
- **Auth Store → Backend API:** POST /api/auth/logout
- **Auth Store → Backend Socket:** Disconnect socket

API Endpoints

Authentication

- GET /api/auth/auth-check – Validate session and return user
- POST /api/auth/login – Authenticate user credentials
- POST /api/auth/signup – Register new user
- POST /api/auth/logout – End session
- PUT /api/auth/update-profile – Update user profile

Messaging

- GET /api/messages/contacts – Retrieve all contacts
- GET /api/messages/chats – Retrieve chat partners
- GET /api/messages/:id – Retrieve message history with a user
- POST /api/messages/send/:id – Send a message to a user

Socket Events

- getOnlineUsers – Broadcast list of online users
- newMessage – Deliver new message to receiver in real time

Resources & Links

- **Screenshots, Demo Videos, Sequence Diagram** [Google Drive Folder](#)

- **Live Website** <https://realtime-chat-application-9-tfrq.onrender.com>